Data structures review

William Hendrix

Lecture 4

Outline

- Review
  - Recursive analysis
  - Stacks and queues
- Sets
- BSTs
- Hash tables
- Maps
- Heaps
- Union-Find

Review: recursive analysis

1. Find recurrence
   - E.g., $T(n) = 2T(n/2) + \Theta(n)$
   - Recursive calls: T terms
   - Other code: O or Θ term

- **Master Theorem**
  - If $T(n) = aT(n/b) + f(n)$:
  - $c = \log_b(a)$
  - Case 1: n^c is polynomially larger than $f(n)$
    - $T(n) = \Theta(n^c)$
  - Case 2: $n^c = \Theta(f(n))$
    - $T(n) = \Theta(n^c \lg(n))$
  - Case 3: n^c is polynomially smaller than f(n)
    - and $f(n)$ is regular
    - $T(n) = \Theta(f(n))$

Review: recursion trees and stacks

- **Recursion tree analysis**
  1. Sketch tree
     - Start with n
     - Add children based on recursive terms
     - "..." then base cases
  2. Analyze tree
     - Apply nonrecursive complexity to each node
     - Add up total
       - May group by level
- **Stacks**
  - Last-In, First Out (LIFO) data structure
  - `push(x)`: add to top of stack
  - `pop()`: remove from TOS
  - `peek()`: see TOS without removing
- **Queues**
  - First-In, First Out (FIFO) data structure
  - `enqueue(x)`: add to back of queue
  - `dequeue()`: remove from front
  - `peek()`: see front without removing
- Push and enqueue are $\Theta(1)$ *amortized time* if we double capacity when full
- Pop, dequeue, and peek are $\Theta(1)$ worst case

Motivating example

- Data structures are a fundamental component of algorithms
- Data organization *strongly* impacts performance

- **Example:** SelectionSort

```
Input: data: an array of integers to sort
Input: n: the number of values in data
Output: permutation of data such that
        data[1] ≤ ... ≤ data[n]
Algorithm: SelectionSort
for i = 1 to n do
    Let m be the location of the min value in the
      array data[i..n]
    Swap data[i] and data[m]
end
return data
```

  - Heap: takes $\Theta(n)$ time to build and $\Theta(\lg n)$ time to delete min
  - Transforms $\Theta(n^2)$ SelectionSort into $\Theta(n \lg n)$ HeapSort

Sets or dictionaries

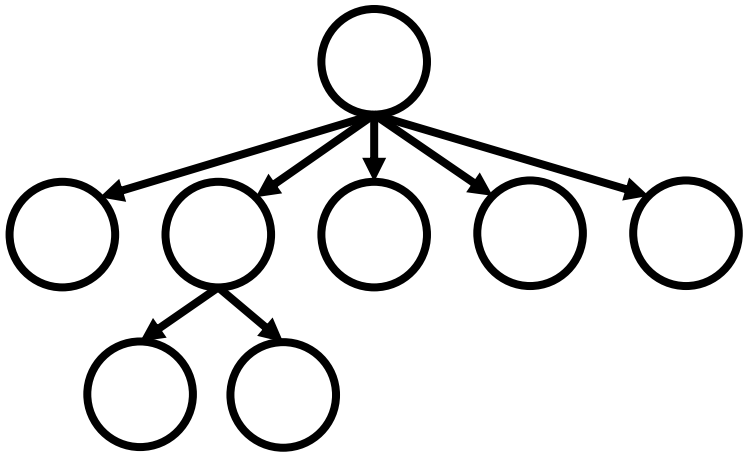
- Abstract data type for storing and retrieving values
  - Generally implemented as hash table or balanced BST
    - Occasionally bit vector

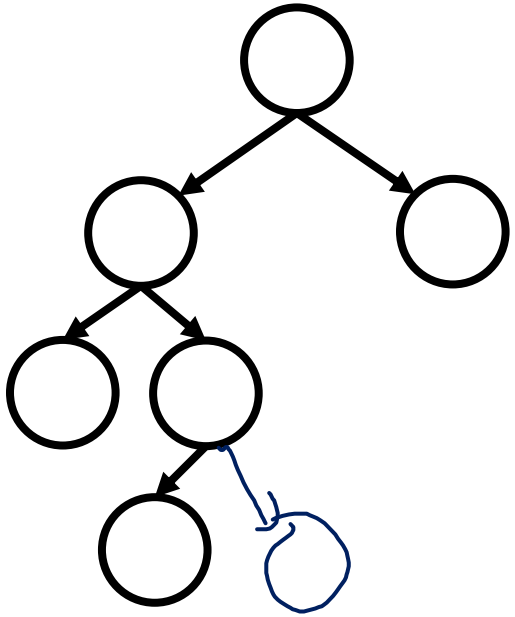
- **Primary operations**
  - *Search(x)*: returns the location of *x* in the set, or NIL if not contained
  - *Insert(x)*: adds *x* to the set
  - *Delete(x)*: removes *x* from the set

- **Additional operations**
  - *Build(data):* construct set from unsorted array
  - *Max(), Min()*: return the location of the largest/smallest element
  - *Successor(x), Predecessor(x)*: return the next largest/smallest element than *x*

Review: trees

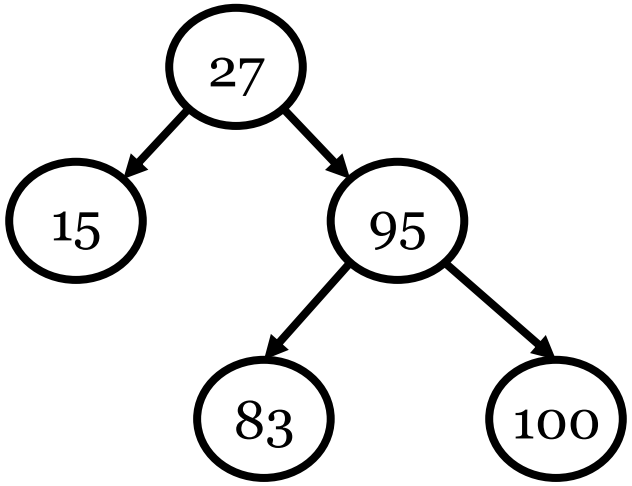
- Nonlinear linked data structure
- From graph theory: any connected acyclic graph
- Most trees in CS are *rooted*
  - Special vertex
  - Other vertices are defined in relation to root
  - Root has children, children have children, etc.
- *Leaf*: vertex with degree 1 (no children)
- *Level*: nodes at same distance from root
- *Height*: max level
- *Siblings*: vertices with same parent
- Binary tree
  - Tree where every node has at most 2 children
    - *left* and *right*
  - *m*-ary tree: every node has at most *m* children
  - *Full* binary tree: all non-leaf nodes have 2 children





Binary search trees

- Binary tree that satisfied **BST property**
  - All elements in left subtree are equal or smaller
  - All elements in right subtree are equal or larger
- *Recursive* data structure
  - Each node defines a BST (subtree)
  - Many algorithms/operations are recursive



BST set operations

- **Search(x)**
  - Binary search
  - Start with Search(root, x)
  - *Θ(1)* time per call
  - *Worst case: h* calls (height of BST)
  - *Θ(h) time*

```
Algorithm: Search(node, x)
if node = NIL then
    return NIL
else if node.data = x then
    return node
else if node.data > x then
    return Search(node.left, x)
else
    return Search(node.right, x)
end
```

- **Insert(x)**
  - Binary search
  - Insert as root or call Insert(root, x)
  - *Θ(h) time*

```
Algorithm: Insert(node, x)
if x ≤ node.data then
    if left = NIL then
        left = NewNode(x)
    else
        Insert(left, x)
    end
else
    if right = NIL then
        right = NewNode(x)
    else
        Insert(right, x)
    end
end
```

BST set operations

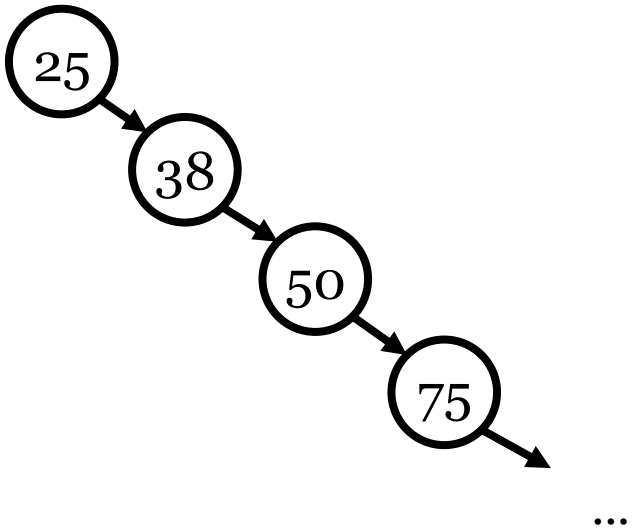
- **Delete(x)**
  - Binary search
  - Special cases depending on children
    - 0 children: delete
    - 1 child: replace w/ child
    - 2 children: find right ST min, swap
  - Worst case analysis
    - *Θ(h)* to find *x*
    - *Θ(h)* to find RST min
    - *Θ(h) total*

```
Algorithm: Delete(node, x)
if node = NIL then
    return
else if node.data > x then
    Delete(node.left, x)
else if node.data < x then
    Delete(node.right, x)
else
    if node.left = NIL and node.right = NIL then
        Set node.parent's child pointer to NIL
        free node
    else if node.left ≠ NIL and node.right ≠ NIL then
        sub = min(node.right)
        Remove sub.parent's child link
        Set sub's 3 links to match node
        Set sub's parent's and child's links to sub
        free node
    else if node.left ≠ NIL then
        Set parent's child pointer to node.right
        node.right.parent = node.parent
        free node
    else
        Set parent's child pointer to node.left
        node.left.parent = node.parent
        free node
    end
end
```

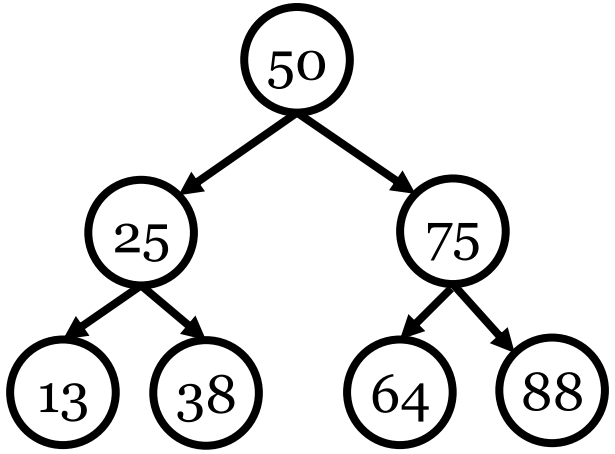
Tree height

- How tall can a BST be?

- **Worst case**
  - Degenerate tree
    - Only left or right children
    - Height = $\Theta(n)$
  - Can happen if inserting in sorted order



- **Best case**
  - Complete tree
    - Each level has twice as many as previous
    - Total nodes: $n = 1 + 2 + 4 + \ldots + 2^h = \Theta(2^h)$
    - $h = \Theta(\lg n)$



Balanced BSTs

- BST variants that *guarantee* $\Theta(\lg n)$ height
  - All ops have great complexity
- Three common balanced BSTs
  - AVL trees
    - Nodes keep track of *balance*
    - Balance = height(*right*) - height(*left*)
    - Force balance to be 0, +1, or -1
  - Red-black trees
    - Nodes are assigned color (red or black)
    - Path to every leaf has same # black nodes
    - No two red nodes in a row
  - Splay trees*
    - Nodes move to the top of the tree as they are requested
- Enforce guarantee by *rotating* tree as elements added and removed
  - We won't cover details

BST complexity

	BST (best)	BST (worst)	Unsorted array	Sorted array
Search(x)	$\Omega(\lg n)$	$O(n)$	$O(n)$	$\Theta(\lg n)$
Insert(x)	$\Omega(\lg n)$	$O(n)$	$\Theta(1)$	$O(n)$
Delete(x)	$\Omega(\lg n)$	$O(n)$	$\Theta(1)$*	$O(n)$
Min()	$\Omega(\lg n)$	$O(n)$	$\Theta(n)$	$\Theta(1)$
Max()	$\Omega(\lg n)$	$O(n)$	$\Theta(n)$	$\Theta(1)$

* Swap to end and decrement size

- BST have great best-case complexity
- Worst case is worse than arrays!

Hash table

- Sparse array-based data structure
- Insert elements according to a *hash function*
  - Function that maps elements in domain to integers 0 to size of array minus one (m-1)
  - Must take $\Theta(1)$ time
- **Example hash function**
  - $f : \mathbb{Z} \rightarrow [0, m-1]$
  - $f(x) = x \bmod m$
    - Overly simple
    - Most hash functions use modulus to ensure range
- **Example**
  - Size = 10, hash function: mod 10
  - Insert 3, 15, 27, 82, 96, 100

100		82	3		15	96	27		

Collisions

- What do we do when two values map to the same location?

100		82	3		15	96	27		

25

- Insert 25
- Separate chaining
  - Each location is the head of a linked list
  - Append new element to list
  - Never "need" to reallocate
- Open addressing
  - Find the next open location, insert there
    - Use *probing strategies* to avoid *congestion*
  - No links, so table can be larger with same memory
  - Deleting elements is tricky
  - Need to manage capacity carefully
- Both scan to find element
  - And periodically double size (*rehashing*)

Operations

- **Search(x)**
  - Hash element
  - Scan linked list (or until empty location)
  - Worst case: *Θ(n)*
- **Insert(x)**
  - Hash element
  - Append to linked list (or scan for open location)
  - Worst case: *Θ(1)* (or *Θ(n)*)
- **Delete(x)**
  - Hash element
  - Delete from linked list (or mark as deleted)
  - Worst case: *Θ(n)*

```
Algorithm: Search(x)
loc = hash(x)
return table[loc].Search(x)
```

```
Algorithm: Insert(x)
ins = new HashNode(x)
loc = hash(x)
ins.next = table[loc]
table[loc] = ins
```

```
Algorithm: Delete(x)
loc = hash(x)
node = table[loc]
if node.value = x then
    table[loc] = node.next
    free node
else
    while node.next ≠ NIL do
        next = node.next
        if next.value = x then
            node.next = next.next
            free next
        node = node.next
    end
end
```

Hash table complexity

Operation	Separate chaining	Open addressing	Balanced BST
Search(x)	$O(n)$	$O(n)$	$O(\lg n)$
Delete(x)	$O(n)$	$O(n)$	$O(\lg n)$
Insert(x)	$O(1)$	$O(n)$	$O(\lg n)$
Build	$O(m + n)$	$\mathbf{O(m + n^2)}$	$O(n \lg n)$
Resize	$O(m + n)$	$\mathbf{O(m + n^2)}$	*n/a*
Min()	$O(m + n)$	$O(m)$	$O(\lg n)$
Max()	$O(m + n)$	$O(m)$	$O(\lg n)$
Predecessor(x)	$O(m + n)$	$O(m)$	$O(\lg n)$
Successor(x)	$O(m + n)$	$O(m)$	$O(\lg n)$

m: capacity
n: size

- This is <u>awful</u>!
- Why would anyone ever use a hash table?

Why would anyone use a hash table?

- Bad worst-case complexity but great *expected-case* complexity
- Expected-case assumptions
  - Hash function produces $\Theta(1)$ collisions
    - Each inserted value has $\Theta(1)$ duplicates
  - $m = \Theta(n)$

- Search(x)
  - Hashing and scanning take $\Theta(1)$ time
- Insert(x)
  - Hashing and scanning take $\Theta(1)$ time
- Delete(x)
  - Hashing and scanning take $\Theta(1)$ time
  - Reinsertion takes $\Theta(1)$ time (open addressing)

Expected-case complexity

Operation	Separate chaining	Open addressing	Balanced BST
Search(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(\lg n)$
Delete(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(\lg n)$
Insert(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(\lg n)$
Build	$\Theta(n)$	$\Theta(n)$	$\Theta(n \lg n)$
Resize	$\Theta(1)$, *amortized*	$\Theta(1)$, *amortized*	*n/a*
Min()	$\Theta(n)$	$\Theta(n)$	$\Theta(\lg n)$
Max()	$\Theta(n)$	$\Theta(n)$	$\Theta(\lg n)$
Predecessor(x)	$\Theta(n)$	$\Theta(n)$	$\Theta(\lg n)$
Successor(x)	$\Theta(n)$	$\Theta(n)$	$\Theta(\lg n)$

- This is <u>amazing</u>!

Set summary

Operation	Balanced BST (worst case)	Hash table (expected)	Separate chaining (worst case)
Search(x)	$O(\lg n)$	$\Theta(1)$	$O(n)$
Delete(x)	$O(\lg n)$	$\Theta(1)$	$O(n)$
Insert(x)	$O(\lg n)$	$\Theta(1)$*	$\Theta(1)$*

- Open addressing: same as separate chaining except that worst case insertion is $O(n)$
- Hash table expected complexity better than balanced BST
  - Worst case is <u>much</u> worse
- Hash tables better "most of the time"
- Balanced BST better if you need to access values in sorted order
- Or if worried about worst case
  - Not sure about hash function or input data
  - Lots of duplicates

Maps

- Abstraction of a function
- Main operations
  - **Insert(x, y):** declares that $f(x) = y$
  - **Delete(x):** declares that $f(x)$ does not have a value
  - **Search(x):** returns y such that $f(x) = y$, or NIL if $f(x)$ does not have a value
- **Example:** letter frequencies
  - Problem: count how many times a letter appears in a given text
    - Used in cryptography
  - Sample output

E	T	A	O	I	N	S	R	H	…
12	9	8	7	7	6	6	6	6	…

  - Need to associate count with letter
  - f(E) = 12, etc.

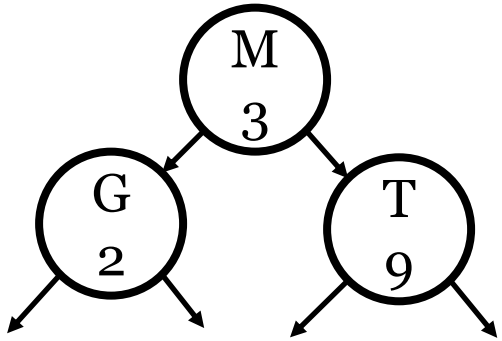
Map implementation

- Two main implementations
- Array-based map
  - Array of all possible x values
  - Stores $f(x)$ in arr[x] (NIL if not initialized)

8	2	3	4	12	2	2	…
A	B	C	D	E	F	G	…

  - All main operations are constant time
  - Only useful when input domain is small

- Set-based map
  - A.k.a., hash map
  - Set of ordered (x, y) pairs
  - Pairs added/searched according to x value
  - **Search** returns associated y value
  - Time complexity determined by hash table or BBST



Map complexity

Operation	Array map	Hash map	Tree map
Insert(x, y)	$\Theta(1)$	$\Theta(1)$*	$O(\lg n)$
Delete(x)	$\Theta(1)$	$\Theta(1)$*	$O(\lg n)$
Search(x)	$\Theta(1)$	$\Theta(1)$*	$O(\lg n)$
Build()	$\Theta(D)$	$\Theta(n)$	$O(n \lg n)$

D: size of domain (*x* values)
* Expected complexity for hash table

The power of maps

- Maps are very useful for storing values that we compute repeatedly
  - Especially when we can use direct maps
- **Example:** Discrete Fourier Transform
  - Given array *x* compute transformed array *c* such that

$$c_k = \sum_{j=1}^{n} x_j \boxed{e^{jk(-2\pi i/n)}}$$

Store values in lookup table

- Can also improve best-case performance for <u>any</u> algorithm

1. Build a map that contains problem instances and solutions
2. Before running another algorithm, test whether input is in map
3. If so, return the answer

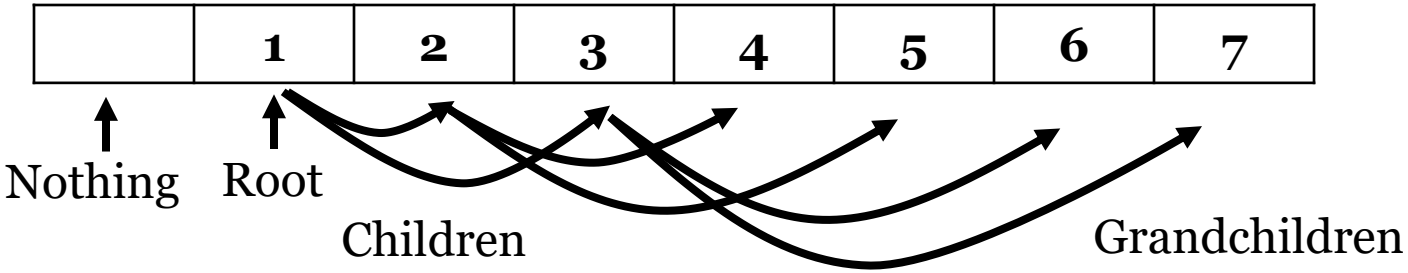
- Best case typically constant or linear time
- Best case analysis not useful to compare algorithm quality

Priority queues

- Abstract data type for finding maxima or minima
- Main operations
  - **Insert(x)**: adds a value to the priority queue
  - **Max()/Min()**: returns max/min value
  - **DeleteMax()/DeleteMin()**: deletes and returns the max/min value from the priority queue
  - Analogous to *enqueue*, *dequeue*, and *peek*
- Primary implementation: heap
  - Complete binary tree (except maybe bottom level)
  - Heap property
    - Every child has value ≤ its parent's value (max-heap)
    - Every child has value ≥ its parent's value (min-heap)
    - Consequence: root node is max (min)
  - Extra op:
    - **Heapify(arr)**: builds a heap from an unsorted array

Heap implementation

- Implemented as array
- Root stored at index 1
- Children of element i stored at $2i$ and $2i+1$
  - Parent at floor($i/2$)



- Because heaps are complete, array has no "gaps"

Priority queue operations

- Descriptions assume max-heap
- **Insert(x):**
  - Append to array
  - Swap with parents until parent is larger (or at root)
  - $\Theta(\lg n)$ time
- **Max():**
  - Return root
  - $\Theta(1)$ time

Priority queue deletion

- **DeleteMax()**
  - Swap max with last element and call PercolateDown(1)
  - **PercolateDown(i)**
    - If *heap[i]* smaller than max child, swap
    - If so, repeat until larger than max child or a leaf
    - $\Theta(\lg n)$ time
  - $\Theta(\lg n)$ time

- **Heapify()**
  - Call PercolateDown(i) from end to beginning
  - Half have no children, quarter have 1 child, etc
  - $\Theta(n)$ time total

```
Algorithm: Delete(x)
Swap heap[1] and heap[n]
n = n − 1
PercolateDown(1)
```

```
Algorithm: PercolateDown(i)
if 2i ≤ n then
    mc = 2i
    if mc + 1 ≤ n and
      heap[mc + 1] > heap[mc] then
        mc = mc + 1
    end
    if heap[i] < heap[mc] then
        Swap heap[i] and heap[mc]
        PercolateDown(mc)
    end
end
```

```
Algorithm: Heapify(i)
for i = ⌊n/2⌋ to 1 step −1 do
    PercolateDown(i)
end
```

Priority queue implementations

Operation	Heap	Unsorted array	Sorted array	Balanced BST	Fibonacci heap
Insert(x)	$\Theta(\lg n)$	$\Theta(1)$	$\Theta(n)$	$O(\lg n)$	$\Theta(1)$
Max()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
DeleteMax()	$\Theta(\lg n)$	$\Theta(n)$	$\Theta(1)$	$O(\lg n)$	$\Theta(\lg n)$, amort.
Build/Heapify	$\Theta(n)$	$\Theta(n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n)$

- BST has similar complexity, but higher coefficients
- Great at finding max (or min)
- Other operations (min/max, search, predecessor, etc.) are not good
  - Min-max heap can do either, but is more complex
- Fibonacci heap has even better complexity
  - More complex, higher coefficients, less space efficient
  - Fairly slow unless data is quite large

Heap exercise

1. Draw the max heap that is built from the array {8, 4, 6, 9, 2, 7, 1}.
2. Draw this heap at the end of every iteration of the **for** loop in HeapSort (below). You may ignore the effect of line 5.

```
   Input: data: an array of integers to sort
   Input: n: the number of values in data
   Output: permutation of data such that
           data[1] ≤ ... ≤ data[n]
1  Algorithm: HeapSort
2  data = MaxHeap.Build(data)
3  for i = n to 2 step −1 do
4  |   m = data.DeleteMax()
5  |   data[i] = m
6  end
7  return data
```

Union-Find data structure

- A.k.a., disjoint set data structure
- **Purpose:** represent partition of dataset
  - Identify whether elements belong to the same subset or not

- **Operations**
  - Initialize(n): set up each element ($1..n$) in its own subset
  - Find(x): return a partition ID for a given element
  - Union(x, y): combine subsets containing x and y together

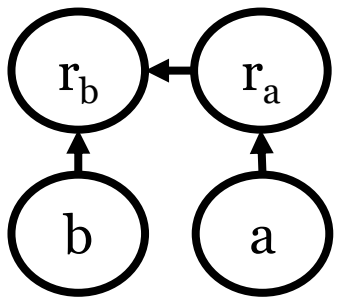
- **Representation:** array of "pointers" (integers)
  - Find: follow pointers until you find a self-loop
    - Self-loop is partition ID ("root")
  - Union: point Find(a) to b

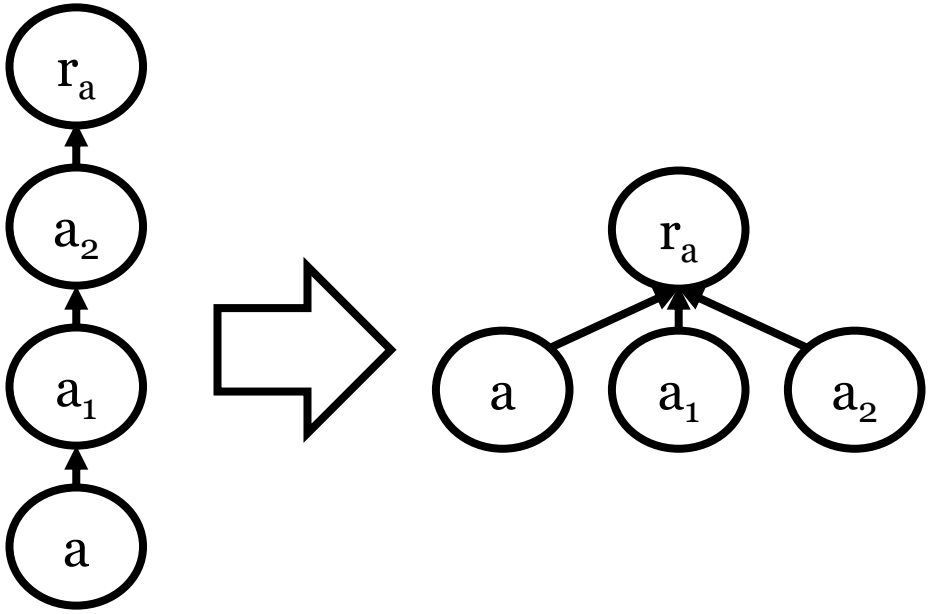
Union-Find example

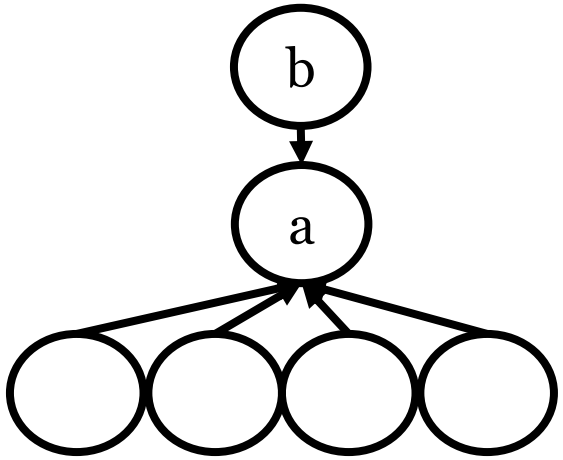
- Show how the data structure changes after each of the following iterations:
  - Initialize(7)
  - Union(1, 2)
  - Union(1, 4)
  - Union(6, 7)
  - Union(3, 5)
  - Union(3, 6)
  - Find(3)

Optimizing Union-Find

- Union complexity depends on Find
- Find complexity depends on height of tree
- Worst case: $\Theta(n)$
- First idea: add Find(a) to Find(b) (or vice versa)
- Second idea: add the smaller tree to the larger
- Third idea: flatten structure when we call Find()







Optimized example

- Show how the data structure changes after each of the following iterations:
  - Initialize(7)
  - Union(1, 2)
  - Union(1, 4)
  - Union(6, 7)
  - Union(3, 5)
  - Union(3, 6)
  - Find(3)

Coming up

- Union-Find
- Sorting algorithms
- Selection algorithms

- **Recommended readings:** Sections 4.1, 4.2, 6.1, 6.2, and 6.3
  - Consider reading section 6.4
  - *Practice problems:* R-4.1, R-4.6, C-4.5, C-4.7, A-4.1, A-4.2, R-6.1, R-6.2, R-6.4 to R-6.7, C-6.6, A-6.1, A-6.6